Ders 9 Çalışma Özeti

Ders 9 Çalışma Özeti

Genel Konular

- Dersin tarihi 4 Kasım 2020 Çarşamba. Ders Zoom üzerinden yapıldı. Hoca dönemin beşinci haftası olduğunu söylüyor.
  - Bir önceki hafta (28 Ekim) yarım gün olduğu için ders yapılamadı.
  - Ders yine "2 saat konu + 1 saat uygulama" düzeninde. Bu kayıt ilk iki saattir.
- Kaba kuvvet (brute force) yaklaşımı ve örnekleri
  - a^n hesaplama, EBOB (gcd), seçmeli sıralama (selection sort), kabarcık sıralama (bubble sort), sıralı arama (sequential search), metin eşleme (string matching), en yakın nokta çifti (closest pair), dışbükey zarf (convex hull).
- Kapsamlı arama (exhaustive search): gezgin satıcı problemi (TSP), sırt çantası problemi (knapsack).
- Azalt ve yönet (decrease and conquer) yaklaşımının üç türü:
  - Sabit miktarda azaltma (decrease by a constant).
  - Sabit oranda azaltma (decrease by a constant factor).
  - Değişken miktarda azaltma (variable size decrease).
- İkili arama (binary search): sözde kod ve en kötü durum analizi.
- Böl ve yönet (divide and conquer) yaklaşımı ve Master teoremi.
- Merge sort ile quick sort'un karmaşıklık karşılaştırması ve kararlılık (stability).

Hocanın Özellikle Vurguladığı Kısımlar

- **Ödev duyurusu:** Cuma sabahı bir ödev verilecek. Teslim süresi Cumartesi gece yarısı. Değişiklik olmazsa iki soru olacak.
  - Ödev Online YıldızEdu üzerinden paylaşılacak. Farklı bir yükleme yeri olursa e-postayla duyurulacak.
  - Ödev koşulları ve kontrol yöntemi ödev dokümanında yazacak.
- Kaba kuvvet her zaman çalışır ama genellikle en verimli çözüm değildir. Yine de bazen diğer yaklaşımlardan daha akıllıca olabilir.
- Kapsamlı aramada çözüm sayısı hızla büyür.
  - 10 eşyalı sırt çantasında $2^{10} = 1024$ alt küme denenir. 100 eşyada 2^{100} olur ve pratikte çözülemez.
  - Bu tür kombinatorik problemler çoğunlukla NP-zor (NP-hard) problemlerdir. Dönemin son haftasında işlenecek.
- İkili arama yalnız sıralı dizide çalışır.
- Master teoremindeki parametrelerin anlamı:
  - n: girdi boyutu.
  - b: problemin kaç parçaya bölündüğü.
  - a: bu parçalardan kaç tanesinin çözülmeye devam ettiği. İkili aramada $a = 1$, merge sort'ta $a = 2$.
  - $f(n)$: bölme ve birleştirme için harcanan iş.
- Böl ve yönet her zaman en iyi çözüm değildir. Özyinelemeli çağrıların bir maliyeti vardır.
- Derse katılım: Hoca yaklaşık 15 öğrencinin adını okuduğunu, çoğunun cevap vermediğini söyledi. Bu konuda son kez uyardığını, adıyla soru sormaya devam edeceğini belirtti.
  - Soru sormak için el kaldırma özelliğini kullanın.

Kısa Tekrar Notları

- Kaba kuvvet: problemi tanımından doğrudan, optimizasyon yapmadan çözmek.
- Kapsamlı arama: kombinatorik problemde bütün olası çözümleri tek tek denemek.
- Azalt ve yönet ile a^n:
  - Birer azaltma: $a^n = a^{n-1} \cdot a$, $a^0 = 1$.
  - Yarıya azaltma: n çiftse $a^n = (a^{n/2})^2$, n tekse $a^n = (a^{(n-1)/2})^2 \cdot a$, $a^0 = 1$.
- Değişken miktarda azaltma örneği: Öklid algoritması, $\gcd(m, n) = \gcd(n, m \bmod n)$.
- İkili arama en kötü durum: $C_w(n) = C_w(\lfloor n/2 \rfloor) + 1$, $C_w(1) = 1$. Sonuç $C_w(n) = \lfloor \log_2 n \rfloor + 1 = \lceil \log_2(n+1) \rceil \in \Theta(\log n)$.
- Genel böl ve yönet bağıntısı: $T(n) = a\,T(n/b) + f(n)$.
- Master teoremi ($f(n) \in \Theta(n^d)$, $d \geq 0$):
  - $a < b^d$ ise $T(n) \in \Theta(n^d)$.
  - $a = b^d$ ise $T(n) \in \Theta(n^d \log n)$.
  - $a > b^d$ ise $T(n) \in \Theta(n^{\log_b a})$.
- Merge sort: her durumda $\Theta(n \log n)$, ek bellek ister, kararlıdır.
- Quick sort: en iyi ve ortalama $\Theta(n \log n)$, en kötü $\Theta(n^2)$, kararlı değildir.

Detaylı Açıklamalar

Kaba kuvvet örnekleri

- a^n**:** a'yı n kez çarpmak.
- **EBOB:** Sayıların ortak bölenlerini tek tek deneyip en büyüğünü bulmak. Öklid algoritması ise ayrı bir yaklaşımdır.
- **Seçmeli ve kabarcık sıralama:** İkisi de en kötü durumda $\Theta(n^2)$. Dizinin sıralı ya da karışık olması yaklaşımı değiştirmez.
- **Sıralı arama:** Bütün elemanlara tek tek bakmak. Daha hızlı arama yöntemleri (hashing gibi) ileride işlenecek.
- **Metin eşleme:** Aranan deseni metnin her konumuna hizalayıp karakter karakter karşılaştırmak. Uyuşmazlıkta deseni bir kaydırmak. Biyoinformatikte gen dizilerinde de kullanılır.
- **En yakın nokta çifti:** Her noktanın diğer her noktaya uzaklığını hesaplayıp en küçüğünü tutmak.

Kapsamlı arama

- **Gezgin satıcı problemi:** Bir başlangıç şehrinden çıkıp her şehre bir kez uğrayıp başlangıca dönen en kısa turu bulmak. Kapsamlı aramada bütün turlar denenir. Türkiye'deki 81 il için bu sayı çok büyüktür.
- **Sırt çantası problemi:** Kapasitesi sınırlı bir çantaya, ağırlıkları ve değerleri bilinen eşyalardan toplam değeri en yüksek olan alt küme seçilir. n eşya için 2^n alt küme denenir.

Azalt ve yönet

- Problem, bir alt probleme indirgenir. Alt problemin çözümü genişletilerek asıl problem çözülür.
- **Sabit miktarda azaltma:** Her adımda boyut sabit bir miktar (çoğunlukla 1) azalır. Örnek: $a^n = a^{n-1} \cdot a$.
- **Sabit oranda azaltma:** Her adımda boyut sabit bir oranla (çoğunlukla 2) bölünür. Örnek: $a^n = (a^{n/2})^2$, ikili arama.
  - n tekse fazladan bir a ile çarpmak gerekir.
- **Değişken miktarda azaltma:** Azaltma miktarı adımdan adıma değişir. Örnek: Öklid algoritması. Her adımda azalma miktarı m mod n'ye bağlıdır.
- İleride topolojik sıralama (topological sorting) da bu yaklaşımla işlenecek. Örnek: ön koşullu derslerden ders planı çıkarmak.

İkili arama

```
BinarySearch(A[0..n-1], K)     // A sıralı
  l <- 0; r <- n-1
  while l <= r do
    m <- floor((l + r) / 2)
    if K = A[m] return m
    else if K < A[m] r <- m - 1
    else l <- m + 1
  return -1
```

- Çıktı: aranan elemanın indisi, yoksa -1.
- Bu sürüm yinelemelidir. İkili aramanın özyinelemeli sürümü de vardır.
- Temel işlem: K ile $A[m]$'nin karşılaştırması. Her turda bir kez yapılır.
- En kötü durum bağıntısı ve çözümü:

$$C_w(n) = C_w(\lfloor n/2 \rfloor) + 1, \quad C_w(1) = 1 \;\Rightarrow\; C_w(n) = \lfloor \log_2 n \rfloor + 1$$

Böl ve yönet

1. Problem, mümkünse eşit büyüklükte daha küçük alt problemlere bölünür.
2. Alt problemler (genellikle özyinelemeli olarak) çözülür.
3. Gerekirse alt problemlerin çözümleri birleştirilerek asıl problemin çözümü elde edilir.

Master teoremi örneği

- $T(n) = 2T(n/2) + 1$: $a = 2$, $b = 2$.
- $f(n) = 1 = n^0$ olduğu için $d = 0$.
- $a = 2 > b^d = 2^0 = 1$. Üçüncü durum geçerlidir: $T(n) \in \Theta(n^{\log_2 2}) = \Theta(n)$.
- Master teoremi derste ispatlanmadı. Yalnız nasıl uygulanacağı gösterildi.

Merge sort ve quick sort

- Merge sort'ta bölme $\Theta(1)$, birleştirme $\Theta(n)$ iş yapar.
- Merge sort'un dezavantajı ek bellek ihtiyacıdır. Avantajı kararlı olmasıdır.
- Kararlılık: eşit değerli elemanlar sıralamadan sonra da aralarındaki eski sırayı korur.
  - Heap sort ve quick sort kararlı değildir.
- Bu iki algoritmanın analizi ders 10'da yapıldı.

Ders sonrası notlar

- **Yarıya azaltmalı a^n daha verimlidir:** Hoca kaba kuvvetle a^n hesaplamanın burada daha akıllıca olduğunu, çünkü yarıya azaltmada "değerlerin hepsinin yine hesaplandığını" söyledi. Bu, yazılan formül için doğru değil. $(a^{n/2})^2$ yazıldığında $a^{n/2}$ bir kez hesaplanır ve sonuç kendisiyle çarpılır. Çarpma sayısı $\Theta(\log n)$ olur. Kaba kuvvette ise $n-1$ çarpma yapılır.
  - Python ile 3^{1000} yarıya azaltma yöntemiyle 16 çarpmayla doğru hesaplandı. Kaba kuvvette 999 çarpma gerekir.
  - Hocanın uyarısı yalnız $a^{n/2} \cdot a^{n/2}$ iki ayrı özyinelemeli çağrıyla hesaplanırsa geçerlidir. O zaman çarpma sayısı yine $\Theta(n)$ olur.
- **İkili aramada sol sınır:** Derste `else` kolu için bir öğrenci "L'yi m-1 yapacağız" dedi. Doğrusu `l <- m + 1`. `r <- m - 1` yalnız $K < A[m]$ kolundadır.
- **a, b'den büyük olabilir:** Bir öğrencinin "a asla b'den büyük olamaz değil mi?" sorusuna hoca "öyle gözüküyor" dedi. Bu doğru değil. Örneğin Karatsuba çarpmasında $T(n) = 3T(n/2) + \Theta(n)$, Strassen matris çarpımında $T(n) = 7T(n/2) + \Theta(n^2)$'dir. Bir problem b parçaya bölünüp birden fazla alt problem üzerinde çakışan işler yapılabilir.
- **Sabit oran:** Hoca bir yerde "constant faktörümüz $n/2$" dedi. Sabit oran 2'dir. $n/2$ alt problemin boyutudur.

- **Kaba kuvvet EBOB:** Bilinen kaba kuvvet yöntemi, $t = \min(m, n)$'den başlayıp iki sayıyı da bölen ilk t'yi bulana kadar t'yi azaltmaktır (ardışık tam sayı kontrolü). Öğrencinin ilk söylediği mod alarak yer değiştirme yöntemi Öklid algoritmasıdır, kaba kuvvet değil.
- **İkili arama formülünün kontrolü:** Python ile $n = 1, 2, 7, 8, 100$ için bağıntı 1, 2, 3, 4, 7 verdi. Bu, $\lfloor \log_2 n \rfloor + 1$ ve $\lceil \log_2(n + 1) \rceil$ ile aynıdır.

Not: İsterseniz bu dersin altyazı (.srt) dosyasını Gemini Notebook gibi bir yapay zeka aracına yükleyerek ders hakkında daha detaylı soru-cevaplar yapabilir ve dersi verimli çalışabilirsiniz.